ENSIA

Artificial Intelligence and Data Science

Semester 7

Introduction to Mobile Robotics

Lab Assignments

Lab 2: Introduction to CoppeliaSim and Differential Drive Motion Modeling

---

Academic Year 2026–2027

Instructor: Dr. A. Khelloufi

Contents

Goals **2**

1 CoppeliaSim Simulator **2**
1.1 Installation . . . 2
1.2 First Steps with CoppeliaSim . . . 2
1.3 Tasks: . . . 2
1.4 ROS 2 Interface . . . 3
1.4.1 Installation . . . 4
1.4.2 Testing the the ROS 2 Interface . . . 4

2 Differential Drive Robot Motion Modeling **4**
2.1 Theoretical background . . . 4
2.1.1 Differential-Drive Robot . . . 4
2.1.2 Pose Representation . . . 5
2.1.3 Kinematic Model . . . 5
2.1.4 Pose Integration . . . 6
2.1.5 Odometry . . . 7
2.2 Tasks: . . . 7

3 Submission Requirements **8**

Goals

- Familiarize with CoppeliaSim and its interface with ROS 2 for robotic simulation and control.
- Control a differential-drive robot in open-loop (teleoperation) mode.
- Compute and visualize the robot's odometry, and compare it with the true pose in simulation.
- Analyze odometry errors and understand their impact on robot localization.

1. CoppeliaSim Simulator

CoppeliaSim, formerly known as V-REP is a general purpose robot simulator with integrated development environment. It is based on a distributed control architecture: each object/model can be individually controlled via an embedded script, a plugin, ROS/ROS2 nodes, remote API clients, or a custom solution. CoppeliaSim supports a wide range of functionalities, making it suitable for both educational and research purposes.

1.1 Installation

- Download CoppeliaSim V4.10.0 rev0 Edu (Ubuntu 24.04 LTS) from: CoppeliaSim.
- Unpack the archive, then navigate to the installation folder and run `./coppeliaSim.sh`.
- Add the following environment variable modifying .bashrc:

```
$ echo "export COPPELIASIM_ROOT_DIR=<your coppelia installation path>" >> ~/.bashrc
```

1.2 First Steps with CoppeliaSim

Once you have launched CoppeliaSim application, CoppeliaSim will initialized one default scene. The user is free to open several scenes in parallel. Each scene shares the application window and the dialogs with the other scenes, but only the active scene content will be visible in the application window or the dialogs. The application window will display several key components (see figure 1):

- **Menu Bar:** Contains options for file management, simulation controls, and settings.
- **Model Browser:** Provides quick access to a library of pre-built models. These models include various robotic systems, sensors, manipulators, and other objects commonly used in simulations.
- **Scene Hierarchy:** Displays the objects and components in your current simulation environment, allowing easy navigation and management.
- **3D Viewport:** The central area where you can visualize and interact with your simulation.

1.3 Tasks:

1. Refer to the CoppeliaSim user manual to familiarize yourself with the simulator's features and functionalities: CoppeliaSim User Manual.
2. Explore the available sensors in the simulator:
   (a) Navigate to the `components/sensors` folder in the Model Browser and identify three different types of sensors.

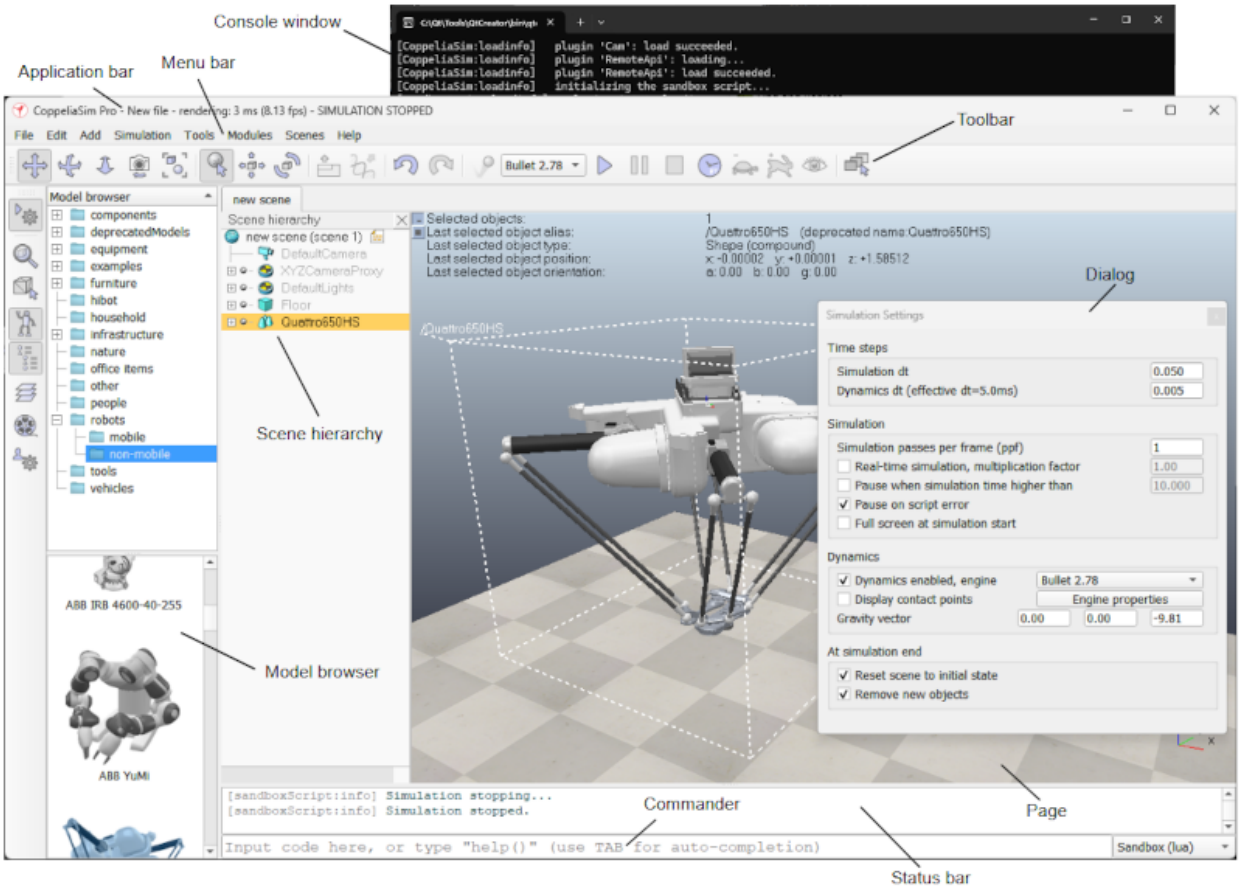


Figure 1: CoppeliaSim user interface.

(b) Briefly describe the functionality of each sensor.

(c) What is the difference between proprioceptive and exteroceptive sensors? Give examples of each.

3. Explore the available wheeled mobile robot models:

   (a) Navigate to `robots/mobile` in the Model Browser.

   (b) Identify three types of wheeled mobile robots.

   (c) Describe the wheel configuration of each robot (e.g., differential drive, Ackermann steering, and omnidirectional drive).

4. Familiarize yourself with the simulator by dragging and dropping a few robot models into the scene, then starting the simulation.

5. Run a few of the demo scenes available in the simulator (File → Open scene...).

6. Follow a few of the tutorials provided with CoppeliaSim.

1.4 ROS 2 Interface

The ROS 2 Interface is part of the CoppeliaSim API framework. It duplicates the C++ ROS 2 API with a good fidelity. This makes it the ideal choice for very flexible communication via ROS 2. CoppeliaSim can act as a ROS 2 node that other nodes can communicate with via services, topics and actions.

For more details see: CoppeliaSimRos2Interface.

1.4.1 Installation

1. Make sure you are in the `ros2_ws/src` directory.

   ```
   $ cd ros2_ws/src
   ```

2. Download the source code. The directory containing all files (i.e. package.xml etc) must be called `sim_ros2_interface`, otherwise build will fail.

   ```
   $ git clone https://github.com/CoppeliaRobotics/simROS2.git sim_ros2_interface
   # according to the docs, we have to rename the folder
   ```

3. Edit the `meta/interfaces.txt` file to include any additional ROS 2 interfaces required by the project. Replace the existing `interfaces.txt` file with the one provided with this lab.

4. Checkout:

   ```
   $ cd sim_ros2_interface
   $ git checkout coppeliasim-v4.10.0-rev0
   ```

5. Return to the root of your workspace and compile:

   ```
   $ cd ~/ros2_ws
   $ colcon build --symlink-install
   ```

   This will take up a while the first time, but don't worry, it will finish and install it.

1.4.2 Testing the the ROS 2 Interface

1. Start CoppeliaSim, load scene `ENSIA_IMR_Lab2.ttt` and start simulation.

2. Running the `ros2 node list` and `ros2 topic list` commands in the terminal to visualize the nodes and topics currently active in the system.

2. Differential Drive Robot Motion Modeling

2.1 Theoretical background

2.1.1 Differential-Drive Robot

A **differential-drive robot** is a mobile robot that has two wheels mounted on either side of a fixed frame. Let:

- r: the radius of each wheel.
- l: the distance between the centers of the two wheels (also called the wheelbase).
- v_R: the linear velocity of the right wheel.
- v_L: the linear velocity of the left wheel.

The velocities of the two wheels v_R and v_L can be used to determine the robot's **linear and angular velocity**:

$$v = \frac{v_R + v_L}{2} \tag{1}$$

$$\omega = \frac{v_R - v_L}{l} \tag{2}$$

where:

- v: linear velocity in the robot's forward direction,
- ω: angular velocity about the robot's vertical axis.

The linear velocities v_R and v_L of the wheels can be expressed in terms of the angular velocities $\dot{\phi}_R$ and $\dot{\phi}_L$ of each wheel:

$$v_R = r \cdot \dot{\phi}_R \tag{3}$$

$$v_L = r \cdot \dot{\phi}_L \tag{4}$$

By substituting v_R and v_L into the equations for v and ω, we can relate the robot's linear and angular velocity to the angular velocities of its wheels.

2.1.2 Pose Representation

In order to represent the motion of a wheeled mobile robot, we must define reference frames and determine their position. There are two essential frames to a robot if we consider the robot as a rigid body:

- **World Frame (I):** also called Inertial frame, the global reference frame where the robot operates.
- **Robot Frame (R):** the local reference frame attached to the robot, with the origin at the robot's center.

The robot's **pose** in the **world frame** is represented by:

$$\xi_I = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

where x and y denote the robot's position in a 2D plane, and θ represents its orientation with respect to the global reference frame.

2.1.3 Kinematic Model

The robot's motion can be modeled using a kinematic model that describes the relationship between the robot's global velocity (expressed in the world frame) and the control inputs (the robot's local velocity expressed in the robot frame).

For a differential drive robot, the robot velocity in the robot frame can be expressed as:

$$\dot{\xi}_R = \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix},$$

To express this velocity in the global frame, we use the rotation matrix $\mathbf{R}(\theta)$. The velocity in the global frame is then:

$$\dot{\xi}_I = R(\theta)\ \dot{\xi}_R.$$

Substituting $\dot{\xi}_I$, $\dot{\xi}_R$, and $\mathbf{R}(\theta)$:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ 0 \\ \omega \end{bmatrix}.$$

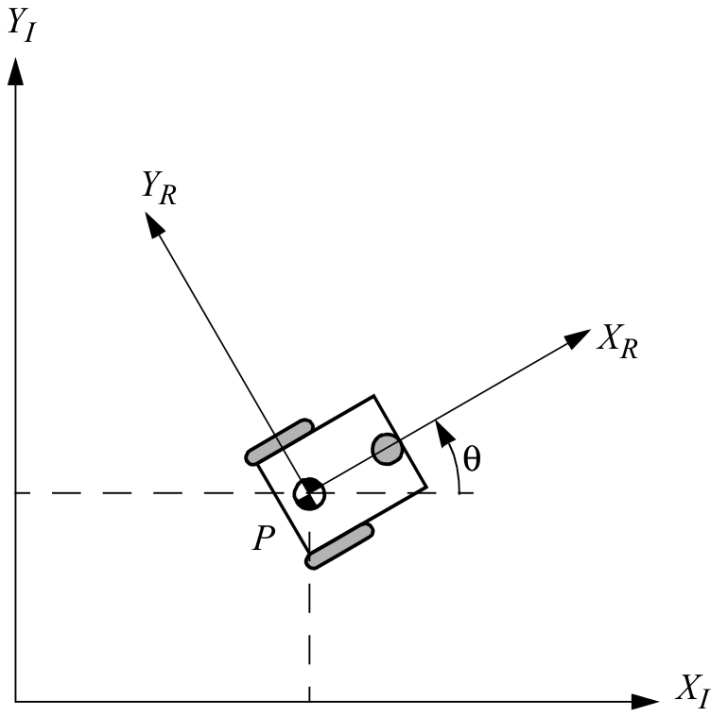


Figure 2: Pose representation in the Inertial frame.

Simplifying:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}.$$

This is the **kinematic model** of a differential drive robot, expressing the relationship between their control inputs and its global motion.

2.1.4 Pose Integration

To compute the robot's pose $\xi_I = \begin{bmatrix} x & y & \theta \end{bmatrix}^T$ over time, we integrate the kinematic equations.

From the kinematic model:

$$\begin{aligned} \dot{x} &= v\ \cos\theta \\ \dot{y} &= v\ \sin\theta \\ \dot{\theta} &= \omega \end{aligned}$$

The continuous-time integrals are:

$$\begin{aligned} x(t) &= x(0) + \int_0^t v(\tau)\ \cos\theta(\tau)\, d\tau, \\ y(t) &= y(0) + \int_0^t v(\tau)\ \sin\theta(\tau)\, d\tau, \\ \theta(t) &= \theta(0) + \int_0^t \omega(\tau)\, d\tau. \end{aligned}$$

In practice, these integrals are computed using numerical methods for discrete time approximation. Indeed, for numerical integration over a time step Δt, the pose update equations are:

$$\begin{aligned} x_{t+\Delta t} &= x_t + v\ \cos\theta_t\ \Delta t, \\ y_{t+\Delta t} &= y_t + v\ \sin\theta_t\ \Delta t, \\ \theta_{t+\Delta t} &= \theta_t + \omega\ \Delta t. \end{aligned} \tag{5}$$

2.1.5 Odometry

In mobile robotics, **odometry** is a method used to estimate the robot's pose over time based on motion data from **proprioceptive sensors**, such as wheel encoders.

Encoders use optical, magnetic, or capacitive principles to detect motion. For example, in optical encoders, light passing through a patterned disk is interrupted to generate pulses proportional to the rotation, which represent wheel spin speed:

1. Encoders count the number of rotations of the robot's wheels.
2. The wheel spin speed $\dot{\phi}_i$ is calculated as:

$$d = \frac{2\pi\ N}{n\ T}$$

   where N is the number of encoder pulses, n is the number of pulses per revolution, and T is the time period (duration over which the encoder counts pulses).

3. Combining the displacements from the left and right wheels enables the calculation of the robot's pose (x, y, θ).

Odometry is susceptible to **cumulative errors** due to wheel slippage and encoder noise, which can cause the pose measurement to drift over time.

The following algorithm **summarize how odometry is calculated** in the **world frame**:

1. **Inputs**:
   - Angular velocities of the left and right wheels $\dot{\phi}_L, \dot{\phi}_R$ (from encoders),
   - Time step Δt,
   - Radius of the wheels r
   - Distance between the two wheels (wheelbase) l
2. **Steps**:
   - Compute v and ω using equations (1) to (4).
   - Update x, y, θ using equation (5).
3. **Repeat**:
   - Integrate over successive time steps to estimate the robot's pose.

2.2 Tasks:

1. Start CoppeliaSim and load scene `ENSIA_IMR_Lab2.ttt`.
2. Use equations (1) to (4) to derive $\dot{\phi}_R$ and $\dot{\phi}_L$, respectively, as functions of v and ω. Then edit the Pionner P3DX Lua script in such a manner that it computes $\dot{\phi}_R$ and $\dot{\phi}_L$ based on the given velocities v and ω.

3. Start the simulation, then return to the terminal and use the command line ros2 topic pub in order to publish the linear and angular velocities to the topic `/cmd_vel`.

   ```
   $ ros2 topic pub --rate 1 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 1.0, y: 0.0,
   z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
   ```

4. Open another new terminal and start the `teleop_twist_keyboard` node. Use the keyboard keys to move the Pionner P3DX robot.

5. Edit the Pionner P3DX Lua script in such a manner that it computes the robot's position and orientation based on the encoder readings and using the above odometry calculation steps.

6. Start the simulation, then return to the terminal and run the `teleop_twist_keyboard` node. Use the keyboard keys to move the Pionner P3DX robot and use the command line `ros2 topic echo` in order to print the calculated odometry (`/odom` topic) and ground truth values (`/pose` topic) to the console.

7. Use the tool `ros2 bag` to record and play data published on topics in your system:
   ```
   $ ros2 bag record -a -o Lab2_<<your name>> # recording all topics
   $ ros2 bag info Lab2_<<your name>> # seeing details about your recording
   $ ros2 bag play Lab2_<<your name>> # playing the bag file
   ```

8. Install **PlotJuggler**, which is a tool used to visualize and plot data:
   ```
   $ sudo snap install plotjuggler
   $ sudo apt install ros-jazzy-plotjuggler-ros
   $ ros2 run plotjuggler plotjuggler
   ```

   Use `PlotJuggler` to plot the robot's pose ($x,\ y,\ \theta$) from `/odom` and `/pose` topics. Plot also the robot path based on odometry and compare it with the ground truth path (from `/pose`). Note that the robot path can be obtained by plotting y in function of x.

9. Play the bag file provided in this lab and use `PlotJuggler` to plot the robot's pose ($x,\ y,\ \theta$) from `/odom` and `/pose` topics, as well as the robot path.

   Compare the odometry-calculated pose with the true pose, and compute the error for the final pose:

$$\text{Position_Error} = \sqrt{(x_{\text{true}} - x_{\text{odom}})^2 + (y_{\text{true}} - y_{\text{odom}})^2}$$

$$\text{Orientation_Error} = \sqrt{(\theta_{\text{true}} - \theta_{\text{odom}})^2}$$

10. Analyze results and conclude.

3. Submission Requirements

1. **Lab Report:** Submit a brief report documenting the work carried out during this lab. The report must include:
   - A screenshot demonstrating the successful installation and compilation of the CoppeliaSim/ROS 2 interface.
   - Answers to all the tasks and questions.
   - Relevant plots illustrating the robot's odometry and ground-truth pose.
   - A discussion and analysis of the obtained results, including the odometry errors.

2. **Updated CoppeliaSim Scene:** Submit the updated `ENSIA_IMR_Lab2.ttt` scene containing the corrected wheel speed equations and odometry calculations implemented in the Pioneer P3DX Lua script.

3. **Teleoperation Video:** Submit a video demonstrating the teleoperation of the Pioneer P3DX robot using ROS 2. The video must show the robot responding to keyboard velocity commands.

   Note: On Ubuntu, you can use the keyboard shortcut `Ctrl + Alt + Shift + R` to start and stop screen recording.